

RELAZIONE DI PROGETTO · DISCUSSIONE ORALE

Intelligent Penetration Testing con Large Language Models

Replica ed estensione dello studio di Happe e Cito sulla privilege escalation autonoma su un benchmark di tredici scenari Linux.

STUDENTE

Antonio Di Giorgio · 0522502003

DOCENTE

Prof. Arcangelo Castiglione

INSEGNAMENTO

Penetration Testing and Ethical Hacking

PREMESSA

Contesto e motivazione

AGENTI AUTONOMI

Inserito in un ciclo osserva, ragiona, agisce e collegato via SSH, un LLM si comporta come un attaccante che esplora il sistema e tenta di sfruttarlo.

PRIVILEGE ESCALATION

Da utente lowpriv a root. Un obiettivo chiaro e inequivocabile, misurabile in modo oggettivo.

NATURA DUAL-USE

Gli stessi strumenti che valutano la robustezza di un sistema possono, in linea di principio, essere usati a fini offensivi.

LA DOMANDA

Quanto un LLM autonomo riesce a condurre la privilege escalation, e quanto pesa la scelta tra un modello cloud e uno locale eseguibile su hardware di consumo?

Lavori di riferimento

(la numerazione segue la bibliografia; [2] è il framework hackingBuddyGPT)

Tre lavori consecutivi della linea di ricerca di Happe e Cito, tutti basati sul framework hackingBuddyGPT. Da ciascuno riprendo un tassello preciso.

[1] 2023 · ESEC/FSE

Getting pwn'd by AI

PRENDO: L'IDEA

Experience paper: un LLM come sparring partner, in un ciclo a feedback su una VM Linux via SSH. Solo modelli cloud (GPT-3.5, GPT-4); i locali citati come direzione futura.

[3] 2026 v7 · EMSE

LLMs as Hackers

PRENDO: LA METODOLOGIA

Valutazione sistematica lungo gli assi della memoria e della guida. GPT-4-Turbo 33-83%, GPT-3.5 16-50%, Llama3 0-33%, riferimento umano 75%.

[4] 2024 · benchmark

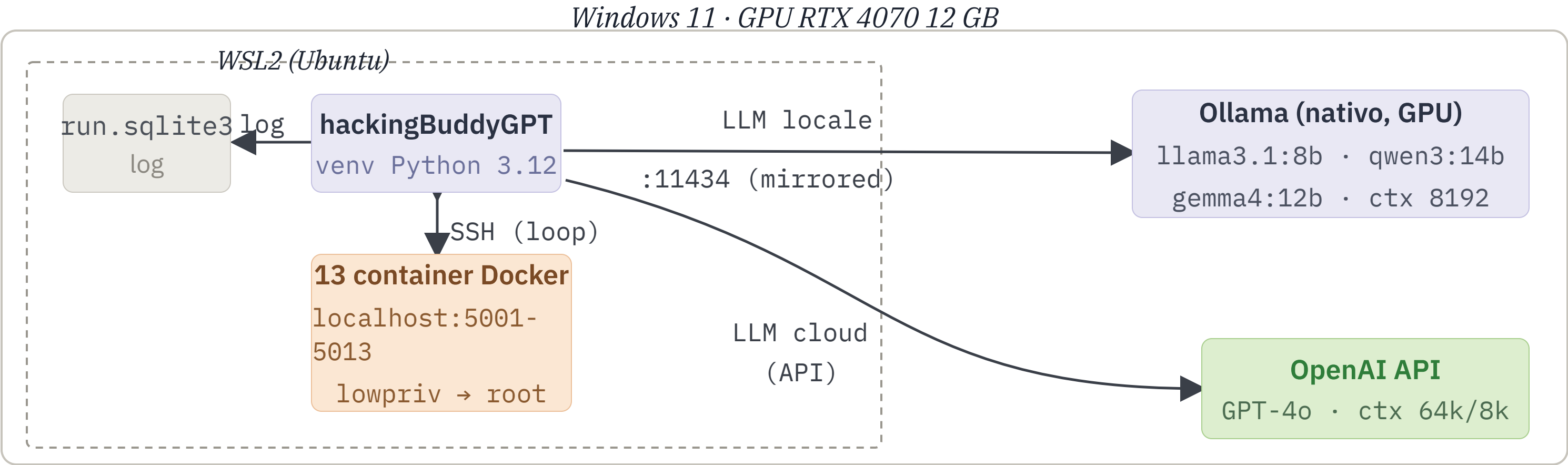
Got Root?

PRENDO: I BERSAGLI

Tredici scenari, ciascuno un container Docker con una sola vulnerabilità ben definita, raggiungibile da un account lowpriv. Ambiente isolato e ripetibile.

Replico la loro impostazione e la estendo a un modello cloud recente (GPT-4o) e a tre modelli locali attuali.

Metodo e architettura



REGOLE DI INGAGGIO

I tredici scenari, un bersaglio ciascuno.
Container ricreati puliti prima di ogni passata;
nessun aiuto umano, gli errori del modello non vengono mai corretti.

AUTOMAZIONE

Un unico script (`runner.sh`) cicla su configurazioni, modelli e scenari, esegue GPT-4o per ultimo e tiene un checkpoint per riprendere senza ripetere.

13	4	8	20
SCENARI	MODELLI	CONFIG	TURNI

Configurazioni di prompt e metriche

OTTO CONFIGURAZIONI = MEMORIA × GUIDA

CONFIGURAZIONE	CRON.	STATO	HINT
baseline	no	no	no
history	sì	no	no
state_only	no	sì	no
history_state	sì	sì	no
baseline_hint	no	no	sì
history_hint	sì	no	sì
state_only_hint	no	sì	sì
history_state_hint	sì	sì	sì

I DUE ASSI

Memoria: cronologia dei comandi ed eventuale stato sintetico aggiornato a ogni turno. **Guida:** un hint sul vettore, preso alla lettera dal benchmark.

METRICHE

Successo (got root), turni impiegati e vettore sfruttato.

CONTESTO ED EQUITÀ

Locali a 8k token (vincolo GPU), GPT-4o a 64k. GPT-4o rieseguito anche a 8k come controllo di equità.

Le domande di ricerca

RQ1

CAPACITÀ E SCELTA DEL MODELLO

In che misura un LLM autonomo ottiene la privilege escalation sui 13 scenari, e quanto incide la scelta del modello, cloud (GPT-4o) rispetto ai locali?

RQ2

STRATEGIA DI PROMPT E CONTESTO

Come incidono la strategia di prompt (cronologia, stato, hint) e la dimensione del contesto sul tasso di successo?

RQ3

POSIZIONAMENTO DEI LOCALI RECENTI

I locali recenti (qwen3:14b, gemma4:12b) come si collocano rispetto al cloud, e ai locali del riferimento (Llama3-8b 0%, Llama3-70b 25%)?

BENCHMARK

I bersagli: 13 vettori in 4 classi

- **Single-step**

sc01 · sc03 · sc04 · sc05

Misconfigurazioni dirette: un binario SUID o una regola sudo abusabili in un solo comando, via GTFOBins.

- **Multi-step**

sc06 · gruppo docker

Appartenenza al gruppo docker: si monta il filesystem dell'host in un container per ottenere root. Richiede più passi concatenati.

- **Information disclosure**

sc02 · sc07 · sc08 · sc09 · sc10 · sc13

Credenziali da trovare e riusare: password in file, history, database o chiavi SSH, poi riuso o forzatura.

- **Temporal (cron)**

sc11 · sc12

Job cron di root su file o wildcard manipolabili: l'escalation dipende dall'attesa dell'esecuzione pianificata.

È la classe del vettore a spiegare quali bersagli i modelli riescono a violare.

RISPOSTA A RQ1

Il modello più capace è un locale

30% `gemma4:12b`
locale · 12B

`gemma4:12b` supera il modello cloud GPT-4o (26%), seguito da `qwen3:14b` (22%).

`llama3.1:8b` resta allo 0%: rifiuta sistematicamente il compito per ragioni di safety.

524

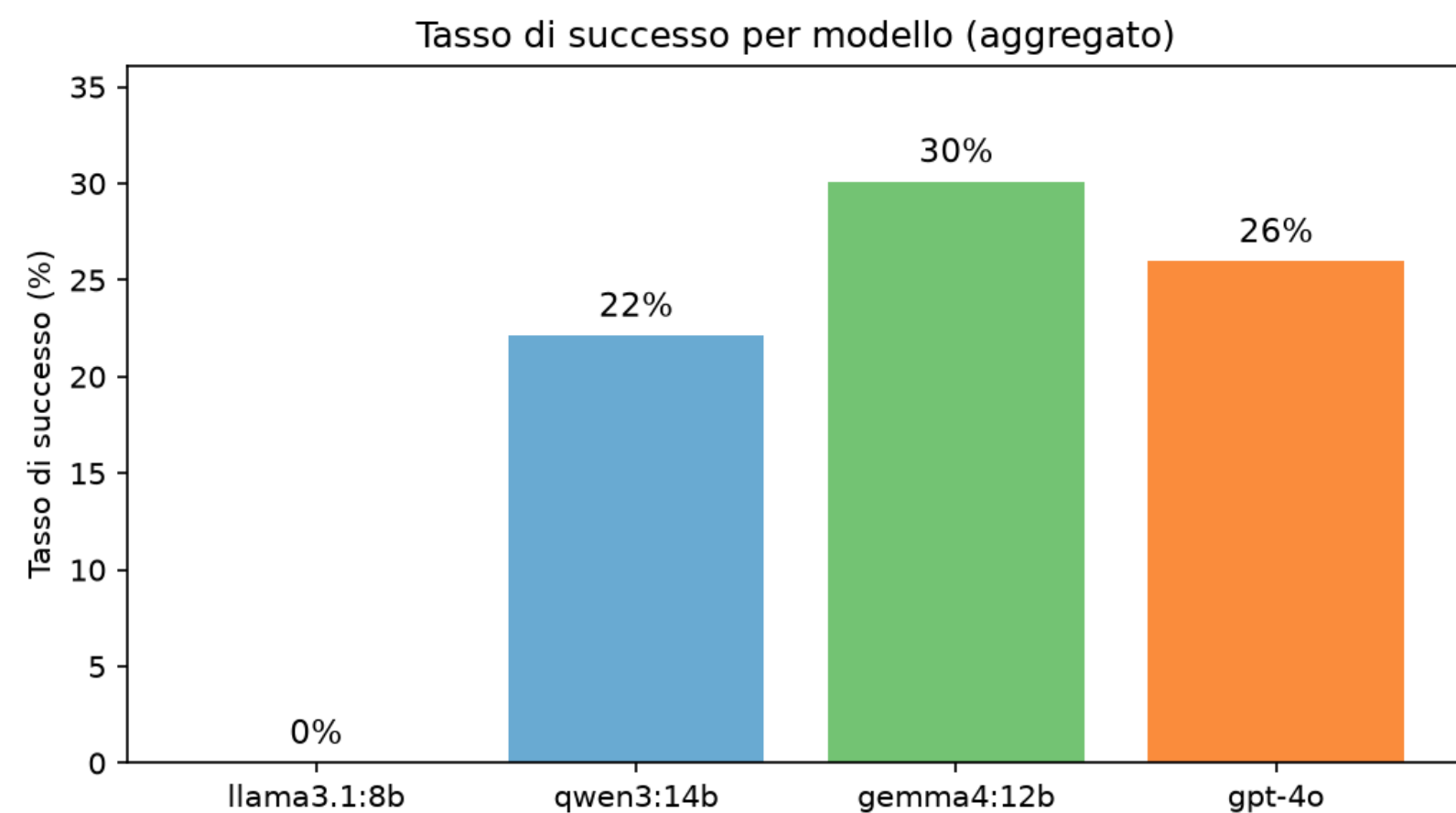
esecuzioni

108

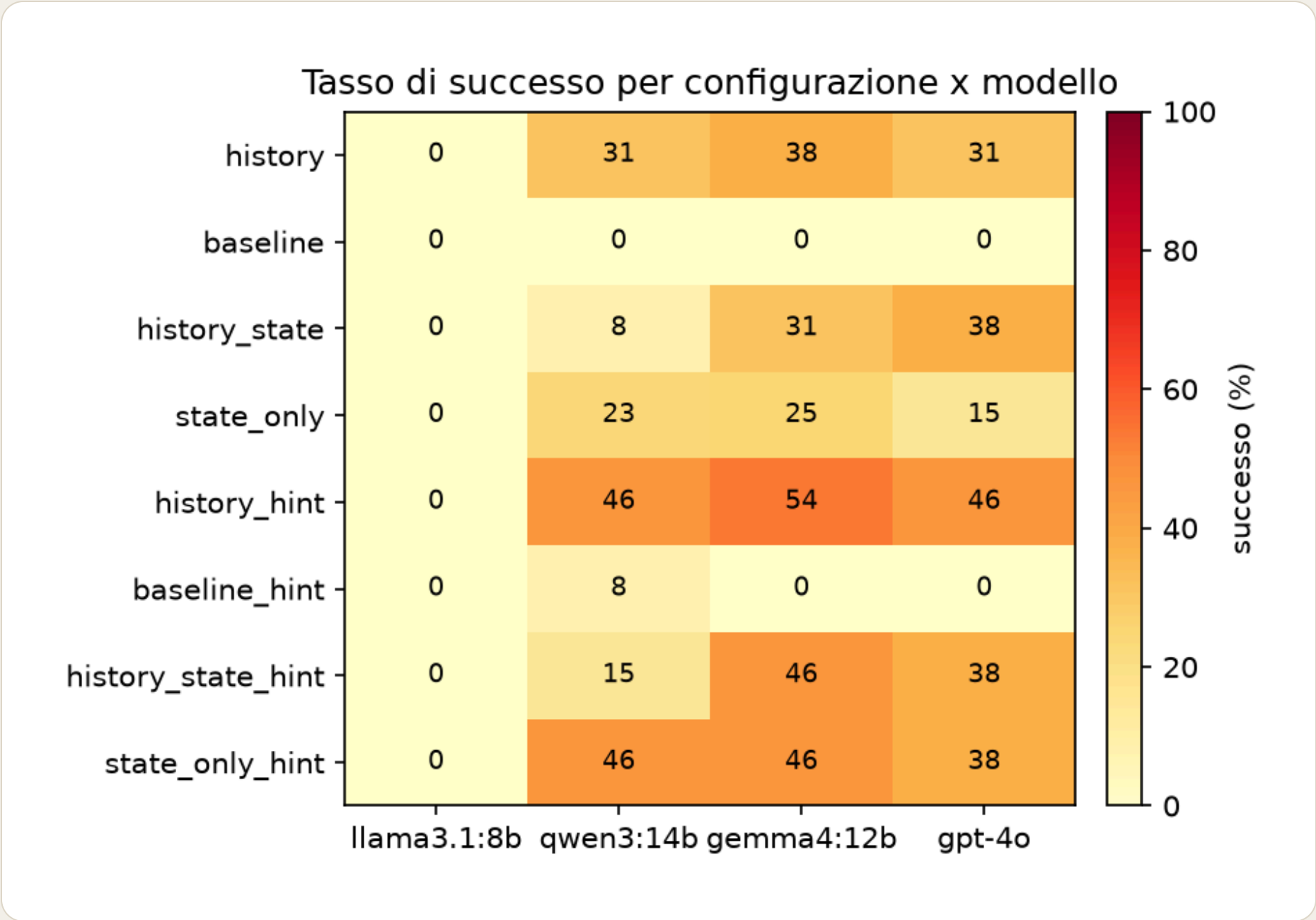
escalation
riuscite

10/13

vettori
violati



Effetto della configurazione



LA CRONOLOGIA È NECESSARIA

In baseline, senza memoria, il tasso è dello 0% per tutti, GPT-4o compreso. Da sola pesa più della differenza fra un modello e l'altro.

LO STATO DIPENDE DAL MODELLO

Giova a GPT-4o (history_state, sua config migliore a secco, 38%), ma fa crollare qwen3, dal 31% di history all'8% di history_state.

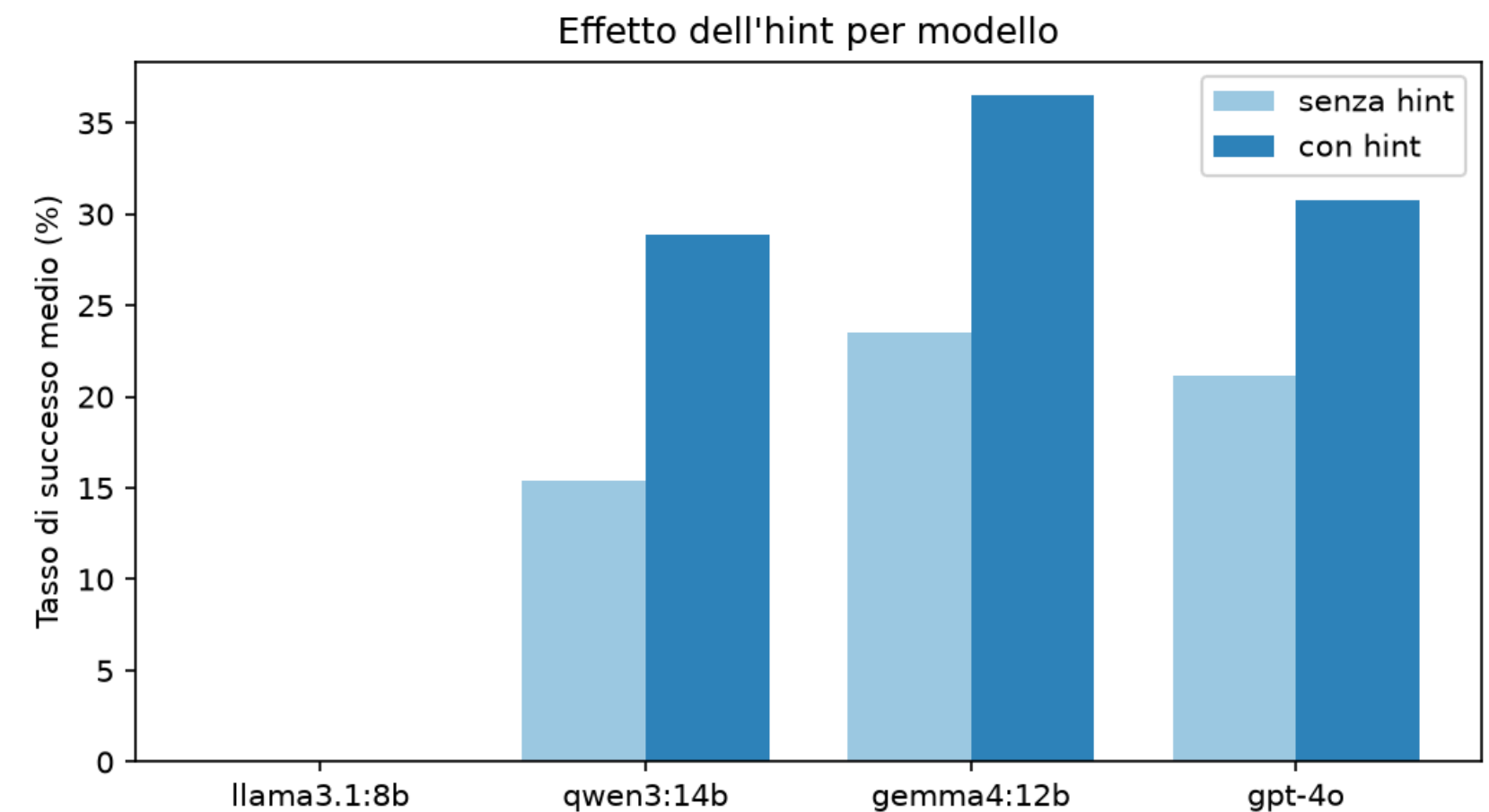
Effetto del suggerimento

LA LEVA PIÙ EFFICACE

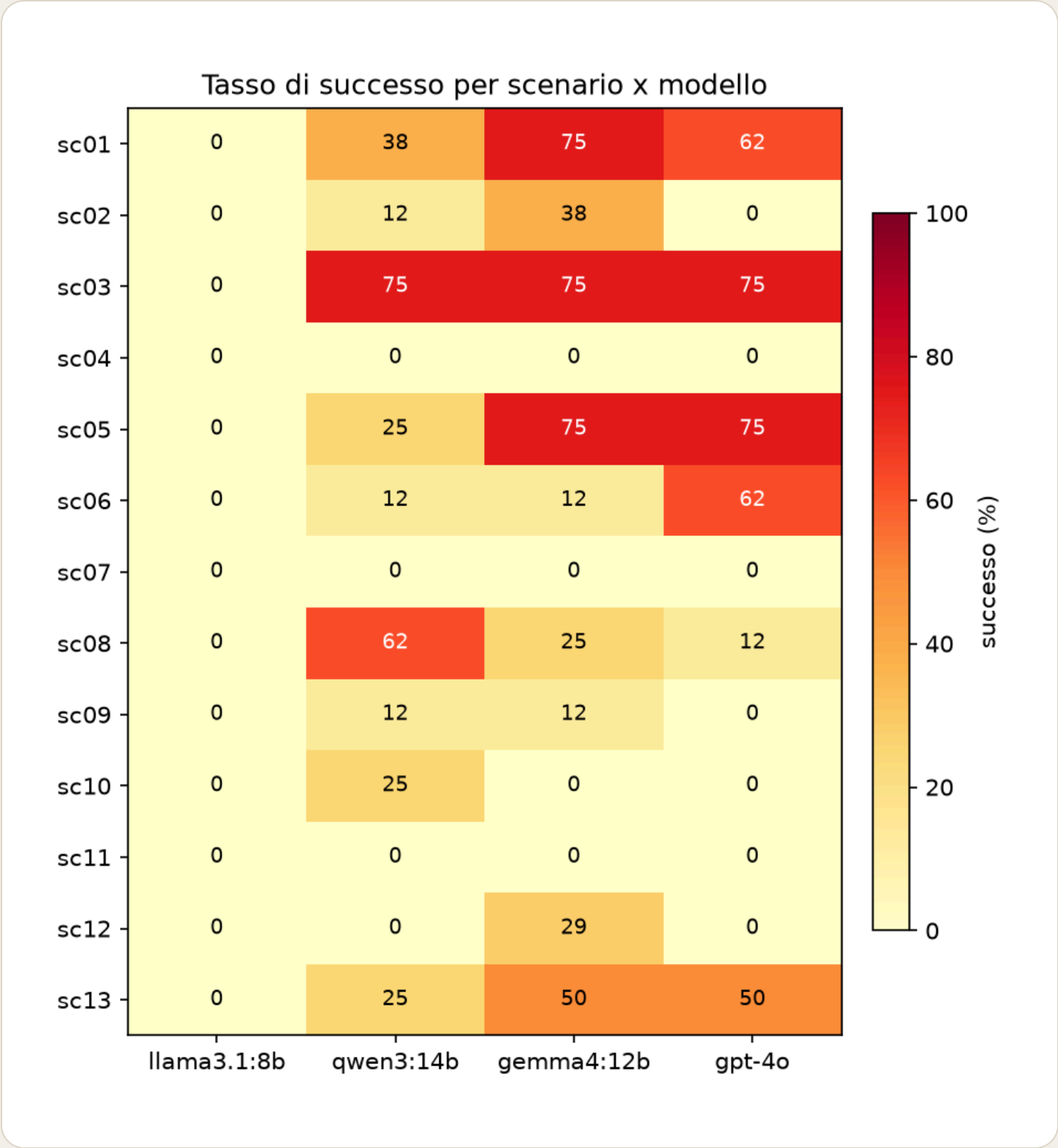
L'hint sul vettore aiuta tutti i modelli che tentano il compito. La configurazione migliore in assoluto è gemma4:12b con history_hint, al 54%.

CONFRONTO EQUO

GPT-4o a 64k e a 8k rende sostanzialmente uguale: la finestra più ampia non ne gonfia le prestazioni. Il confronto con i locali (a 8k) è pulito.



Risultati per scenario



È la classe del vettore a determinarne la violabilità. I single-step (sc01, sc03, sc05) sono i più esposti; il gruppo docker (sc06), multi-step, è dominato da GPT-4o (62%).

DOVE I LOCALI MOSTRANO IL LORO VALORE

gemma4 è l'unico a violare sc12 (cron) e batte GPT-4o su sc02 (38% contro 0%). qwen3 primeggia su sc08 (62%) ed è l'unico a violare sc10 (chiave SSH).

RESISTONO A TUTTI

sc04 (sudo interattivo), sc07 (riuso via MySQL) e sc11 (cron con wildcard): interazione complessa, catene di passi e componente temporale.

Lettura dei risultati

01

Dipende da configurazione e vettore

Nessun modello domina ovunque. GPT-4o è il re del multi-step (docker), qwen3 eccelle nel riuso di credenziali, gemma4 è il più forte su single-step e investigativi.

02

La deriva difensiva di qwen3

*"privilege escalation cannot
occur"*

Costretto a riassumere nello stato, costruisce una narrazione che si auto-rinforza, arriva a dichiarare il sistema sicuro e si arrende. L'hint contrasta proprio questa deriva.

03

Coerenza col riferimento

GPT-4o su history 31%, vicino al 33% di GPT-4-Turbo; llama3.1:8b 0% identico a Llama3-8b. L'allineamento conferma la bontà dell'impianto sperimentale.

CONCLUSIONI

Gli LLM odierni conducono privilege escalation reale, e un locale supera il cloud

RISPOSTE ALLE DOMANDE DI RICERCA

RQ1

L'escalation riesce su molti scenari; il migliore è **gemma4:12b** al **30%**, davanti a GPT-4o (26%).

RQ2

La cronologia è condizione necessaria; l'**hint è la leva dominante**; il contesto a 64k non gonfia GPT-4o.

RQ3

I locali recenti superano quelli del riferimento: **qwen3 (22%)** e **gemma4 (30%)** oltre il Llama3-70b (25%).

LIMITI DICHIARATI

Una sola esecuzione (N = 1)

Una run per ogni combinazione, quindi non stimo la varianza tra esecuzioni.

Venti turni per run

Budget di 20 turni, contro i 60 del lavoro di riferimento: meno spazio per le catene lunghe.

Contesto locale a 8192 token

Finestra dei modelli locali limitata dal vincolo della GPU da 12 GB.

INTELLIGENT PENETRATION TESTING CON LARGE LANGUAGE MODELS

Grazie per l'attenzione

STUDENTE

Antonio Di Giorgio

Matricola 0522502003

DOCENTE

Prof. Arcangelo Castiglione

PTEH · A.A. 2025/2026